Supplementary Materials for: Merkle Tree-Based Inheritance in Decentralized Module Systems

CCS Concepts: • **Software and its engineering** → **Software design techniques**; • **General and reference** → *Evaluation.*

Keywords: Merkle trees, decentralized systems, module inheritance, software design, evaluation

ACM Reference Format:
. 2025. Supplementary Materials for: Merkle Tree-Based Inheritance in Decentralized Module Systems. In . ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

A Formal Properties of Finalization

I now formally state and prove the core semantic properties of my finalization model: **Determinism**, **Injectivity**, and **Soundness**.

A.1 Determinism

Theorem A.1 (Determinism of Finalization)**.** *For all inputs* $(C, T, A, \{r_1, \ldots, r_n\})$*, finalization always produces a unique result. That is:*

$$\mathcal{F}(C, T, A, \{r_1, \ldots, r_n\}) = h_M$$

is a total and deterministic function.

Proof. Each step of $\mathcal{F}$ consists of deterministic subcomputations:

- $\mathcal{R}$ resolves symbolic names to hashes via a fixed registry map.
- Ancestry $A(M)$ is constructed by recursive union over parents, deterministically ordered.
- The Merkle root $H_{\text{merkle}}(P)$ is computed from a lexicographically sorted list, ensuring unique tree layout.
- Content hashing $h_C = H(C)$ and final ID computation $h_M = H(h_C||T||A||H_{\text{merkle}}(P))$ are purely functional.

All operations are total and deterministic, thus $\mathcal{F}$ is deterministic. □

A.2 Injectivity

Theorem A.2 (Injectivity of Finalization)**.** *If* $\mathcal{F}(C_1, T_1, A_1, P_1) = \mathcal{F}(C_2, T_2, A_2, P_2)$*, then:*

$$(C_1, T_1, A_1, P_1) = (C_2, T_2, A_2, P_2)$$

Proof. Assume $h_1 = \mathcal{F}(C_1, T_1, A_1, P_1)$ and $h_2 = \mathcal{F}(C_2, T_2, A_2, P_2)$ and $h_1 = h_2$.

Then:

$$H(h_{C_1}||T_1||A_1||H_{\text{merkle}}(P_1)) = H(h_{C_2}||T_2||A_2||H_{\text{merkle}}(P_2))$$

Since H is a cryptographic hash function with collision resistance, the input tuples must be byte-for-byte identical:

$$(h_{C_1}, T_1, A_1, H_{\text{merkle}}(P_1)) = (h_{C_2}, T_2, A_2, H_{\text{merkle}}(P_2))$$

Thus $C_1 = C_2$, $T_1 = T_2$, $A_1 = A_2$, and P_1 and P_2 have identical Merkle roots. Because the Merkle tree is built over a sorted list of finalized IDs, equality of Merkle roots implies identical parent sets. Hence, inputs are equal. □

A.3 Monotonicity

Theorem A.3 (Monotonicity of Finalization)**.** *Let* $M_1 \sqsubset M_2$ *denote that* M_2 *is a refinement of* M_1 *(i.e.,* $C_1 \subseteq C_2$ *and* $T_1 = T_2$*,* $A_1 = A_2$*,* $P_1 = P_2$*). Then:*

$$M_1 \sqsubset M_2 \Rightarrow H(M_1) \neq H(M_2)$$

but

$$\mathit{ResolutionTable}(M_1) \subseteq \mathit{ResolutionTable}(M_2)$$

A.4 Termination

Theorem A.4 (Termination of Finalization)**.** *Finalization always terminates on well-formed input:*

$$\forall M.\ \mathit{acyclic}(A(M)) \Rightarrow \exists h_M.\ \mathcal{F}(C, T, A, P) \Downarrow h_M$$

Proof. Finalization proceeds in the following bounded steps:

1. **Symbolic resolution** is finite over a finite set of references $\{r_1, \ldots, r_n\}$.
2. **Ancestry expansion** is a recursive set-union traversal over the parent graph. Since $A(M)$ is acyclic and finite (each module has a finite number of parents), the recursion always terminates.
3. **Merkle tree construction** uses a lexicographically sorted list of finalized parent hashes, which is a finite and terminating computation.
4. **Hashing** operations are total and constant-time.

Since all substeps are terminating, $\mathcal{F}$ terminates on any well-formed acyclic input. □

Proof. The final hash includes $H(C)$ as a component. If $C_1 \subset C_2$, then $H(C_1) \neq H(C_2)$ with overwhelming probability (by hash injectivity). Therefore, $H(M_1) \neq H(M_2)$.

For resolution, since M_2 adds only new declarations and does not remove or shadow any from M_1, the computed resolution table extends that of M_1. Resolution proceeds left-to-right along a fixed linearization of A, so new entries in C_2 appear earlier or extend the resolution domain, preserving all bindings from M_1. □

Corollary A.5 (Stability under Extension)**.** *Finalizing a module* M *multiple times with extended content (but fixed ancestry*

and types) produces distinct IDs, but earlier bindings remain valid:

$$\textit{If } M \sqsubset M', \textit{ then } R_M \subseteq R_{M'}$$

A.5 Soundness

Theorem A.6 (Symbol Resolution Soundness)**.** *Let M be a finalized module with resolution table R_M computed at finalization time. Then for every $s \in dom(R_M)$:*

$$M \vdash s \Downarrow d \rightsquigarrow \pi \quad \textit{and} \quad \Gamma \vdash d : \tau$$

and if s overrides a parent symbol s', then:

$$\Gamma(d) <: \Gamma(s')$$

Proof. Resolution uses a fixed order over $A(M)$ defined by Merkle ordering. For each s, the first provider M_i is selected uniquely. By determinism of this ordering, M_i is unique, and so is d.

Type soundness follows from override checks during finalization. For any override, finalization verifies that the overriding type τ' is a subtype of the overridden type τ, satisfying $\tau' <: \tau$. Since typing is preserved under subtyping, all symbol uses remain valid. □

A.6 Corollary: Functional Resolution

Corollary A.7. *Symbol resolution is a partial function:*

$$R_M : name \rightharpoonup (M_i, s_j)$$

and if $M \vdash s \Downarrow d_1$ and $M \vdash s \Downarrow d_2$, then $d_1 = d_2$.

Proof. Immediate from the uniqueness property of Merkle ordering and Theorem A.1. □

B Formal Resolution and Override Rules

B.1 Symbol Resolution and Override Traces

Tangl resolves symbols using a breadth-first search (BFS) traversal over the module inheritance graph, starting from the child and proceeding to its ancestors. This design guarantees predictable symbol resolution across arbitrarily deep module graphs and avoids the ambiguity of linear inheritance.

Resolution Semantics.

- Resolution is performed using a child-first breadth-first search.
- Symbols are identified by their `name` and `scope`.
- If multiple symbols with the same name and scope appear during traversal, only the first encountered (i.e., closest to the child) is retained.
- If a symbol is aliased during import, its resolution respects the alias rather than the original name.
- Only the final resolved set of symbols is stored in the module's public symbol table.

Overrides and Traces.

- Overridden symbols are not discarded: they are tracked via metadata linking each symbol to its parent module and override history.
- A future extension will introduce a dedicated query API for retrieving override chains.
- Currently, override traces require manual traversal of the parent chain using the symbol metadata.

Error Behavior. Resolution failures result in descriptive error messages and occur in the following cases:

- A symbol is referenced that does not exist in any reachable parent.
- Two aliased symbols with the same alias name are imported.
- A malformed import produces an invalid scope or alias.

Cyclic Dependencies. Cyclic inheritance is structurally forbidden in the module graph; therefore, resolution is guaranteed to terminate.

B.2 Resolution Inference Rules

I define a judgment $\Gamma \vdash M \Downarrow \Sigma$ meaning that under module context Γ, the module M resolves to symbol table Σ.

[Resolve-Empty]

$$\frac{}{\Gamma \vdash M[\,] \Downarrow \{\}}$$

[Resolve-Self]

$$\frac{\texttt{sym} \in \text{Symbols}(M)}{\Gamma \vdash M[\texttt{sym}] \Downarrow \{\texttt{name} \mapsto \texttt{sym}\}}$$

[Resolve-Parent]

$$\frac{\Gamma \vdash P \Downarrow \Sigma \quad \texttt{name} \notin \text{dom}(\text{Symbols}(M))}{\Gamma \vdash M \Downarrow \Sigma \cup \text{Symbols}(M)}$$

[Resolve-Alias]

$$\frac{\Gamma \vdash P \Downarrow \Sigma \quad \Sigma(\texttt{name}) = \texttt{sym}}{\Gamma \vdash \texttt{alias name} \mapsto \texttt{sym}}$$

[Resolve-Conflict]

$$\frac{\Gamma \vdash P_1 \Downarrow \Sigma_1 \quad \Gamma \vdash P_2 \Downarrow \Sigma_2 \quad \texttt{name} \in \text{dom}(\Sigma_1) \cap \text{dom}(\Sigma_2)}{\Gamma \vdash \text{error: conflicting symbol definitions}}$$

B.3 Override Trace

Given a resolved symbol `sym` in module M, I define its override trace as the sequence:

$$\texttt{trace(sym)} = [\texttt{sym}_0, \texttt{sym}_1, \ldots, \texttt{sym}_n]$$

where $\texttt{sym}_0$ is defined in M and each $\texttt{sym}_{i+1}$ is the symbol it overrides in M's parent graph.

Currently, this trace must be computed manually by walking the parent chain; future work includes exposing this trace via a query API.

B.4 Symbol Metadata and Override Trace (Formal Version)

Each symbol s carries associated metadata recording:

- Its *origin* module M_s where it was initially defined.
- (Optional) A reference $\text{parent}(s)$ to the immediate symbol it overrides, if any.

Override Relation. I define the *override relation* $\prec$ between symbols:

$$s' \prec s \quad \text{iff} \quad \text{parent}(s) = s'$$

That is, s overrides s' if s' is the parent of s in metadata.

Override Chain (Trace). The *override chain* (or trace) from a symbol s is the sequence $s_0, s_1, s_2, \ldots$ where:

$$s_0 = s$$

$$\forall i \geq 0.\ \text{parent}(s_i) = s_{i+1}$$

and $\text{parent}(s_n) = \varnothing$ for some n, terminating the chain.

Inference Rules for Traversal.

$$\frac{\text{parent}(s) = \varnothing}{s \Downarrow [\,]}\ \textsc{Trace-Term}$$

$$\frac{\text{parent}(s) = s' \qquad s' \Downarrow T}{s \Downarrow s' :: T}\ \textsc{Trace-Step}$$

Where:

- $s \Downarrow T$ means "tracing from s yields trace T"
- $::$ denotes list cons (prepend s' to trace T)

Thus, recursively following parent fields reconstructs the entire override history.

Resolution Failure. Symbol resolution fails if:

- A referenced parent symbol is missing.
- Scope collisions occur without explicit aliasing.

All such failures must emit descriptive error messages identifying the failing symbol and cause.

Properties.

- Override chains are **acyclic** because cyclic inheritance is forbidden in the module graph.
- Every override trace is **finite** because modules form a finite acyclic graph.
- Child-first resolution ensures that the first resolved symbol encountered during BFS is the final stored version.

Listing 1. Recursive Symbol Trace

```
def trace(s):
  if s.parent is None:
    return []
  else:
    return [s.parent] + trace(s.parent)
```

Listing 2. Non-recursive Symbol Trace

```
def trace(s0):
  result = []
  current = s0
  while current is not None:
    result.append(current)
    current = metadata[current].overrides
  return result
```

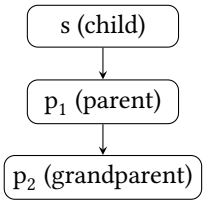

Figure 1. Symbol override chain: each symbol points to its immediate parent.

B.5 Soundness of Symbol Resolution

Theorem B.1 (Soundness)**.** *For any module graph G, symbol s, and resolution request R:*

$$\textit{Resolve}(G, R)$$

either returns a symbol s' or fails with a descriptive error.

Proof Sketch. Resolution proceeds by a breadth-first traversal of G starting from the target node. At each visited node:

- If a matching symbol is found, it is recorded.
- If multiple matching symbols are found, child-first precedence applies.
- If no matching symbol is found after visiting all reachable nodes, an error is emitted.

Because:

1. G is acyclic (by construction),
2. BFS visits all reachable nodes without omission,
3. Conflict resolution rules are deterministic and well-defined,

Resolution must either find exactly one final symbol or detect and report failure. Thus, resolution is sound. □

B.6 Determinism of Symbol Resolution

Theorem B.2 (Determinism)**.** *For any module graph G and resolution request R, repeated applications of $\textit{Resolve}(G, R)$ always produce the same symbol s' or the same error.*

Proof Sketch. Resolution operates by:

- Performing a breadth-first traversal of G,
- Following a fixed child-first precedence order,
- Applying a deterministic symbol matching rule based on name and scope.

Because:

1. The traversal order is uniquely determined by the structure of G,
2. The conflict resolution rules are deterministic,
3. No random or nondeterministic choices are made during resolution,

repeated executions of Resolve(G, R) must yield the same result.

Thus, resolution is deterministic. □

B.7 Compositionality of Symbol Resolution

Theorem B.3 (Compositionality)**.** *Given two module graphs G_1 and G_2, and a merged graph $G = G_1 \cup G_2$, symbol resolution over G behaves as if performing resolution independently over G_1 and G_2 and then combining the results under the conflict rules.*

Proof Sketch. Symbol resolution is based on:

- Breadth-first traversal (BFS),
- Child-first precedence,
- Deterministic symbol replacement by name and scope.

In a merged graph G:

1. Traversal over G can be seen as traversal over G_1 and G_2 in a combined BFS queue,
2. Symbols from G_1 and G_2 are resolved independently until a name conflict occurs,
3. Conflict resolution is deterministic and respects the child-first rule,
4. Thus, symbol resolutions from G_1 and G_2 compose predictably into the merged graph G.

Therefore, the resolution result over G corresponds to the compositional result of resolutions over G_1 and G_2 individually. □

B.8 Summary of Symbol Resolution Properties

Together, the properties of soundness, determinism, and compositionality establish that the symbol resolution mechanism is:

- **Sound**: Every successfully resolved symbol corresponds to a valid and reachable definition.
- **Deterministic**: The result of resolution is uniquely determined by the module graph structure and import declarations, independent of traversal order variations.
- **Compositional**: Resolution results for disjoint or partially overlapping module graphs can be combined predictably under the same conflict and precedence rules.

These properties ensure that module composition is reliable, predictable, and safe, even in the presence of overrides, deep hierarchies, and complex dependency structures.

C Generic Types and Structural Constraints

Though type-agnostic by default, the system accommodates ecosystems with parametric polymorphism and structural typing. This section outlines optional extensions for expressing type-level abstraction and constraints across modules, layered atop the Merkle-based model without modifying its core semantics.

C.1 Generic Type Aliases

Modules may define parameterized type aliases of the form:

$$\texttt{type}\ T\langle X_1, \ldots, X_n \rangle = \tau$$

where τ may reference type parameters X_i. When imported, these aliases are instantiated with concrete types:

$$T\langle \sigma_1, \ldots, \sigma_n \rangle$$

Instantiations are resolved into fully derived types within the importing module.

C.2 Structural Constraints

Type parameters may be constrained by structural predicates:

$$\texttt{type}\ Map\langle K : \texttt{Hashable}, V \rangle = \ldots$$

Here, `Hashable` denotes a required property or interface, transitively resolved through the module graph. Finalization ensures each constraint is satisfied by evidence such as declared instances, structural matches, or subtype edges.

C.3 Finalization Semantics

Generic types are instantiated and expanded during finalization. Constraint satisfaction is resolved transitively via module inheritance, yielding deterministic, content-addressed types. The result must be fully resolved and non-parametric in the final Merkle tree.

C.4 Applications

These mechanisms enable:

- Parameterized types (`List<T>`, `Map<K, V>`).
- Generic algorithms over abstract constraints.
- Interface-based modularity without coupling to implementations.

Use of generics is optional. Ecosystems lacking type-level abstraction can ignore this layer without affecting correctness or resolution semantics.

D Connection to Existing Module Calculi

This system draws on ideas from established module calculi, particularly the ML module system and mixin-based semantics. This appendix sketches the correspondence.

D.1 Structural Modules as Records

Each module can be seen as a finite record of named definitions, possibly parameterized. Resolution involves merging records from parent modules, subject to override rules. This is analogous to record extension and override in the ML module system.

D.2 Inheritance and Override as Mixin Composition

Inheritance corresponds to mixin composition, where multiple modules are combined into one, with later entries overriding earlier ones. If two modules A and B are merged (with B overriding), the result resembles:

$$M = A \oplus B$$

where $\oplus$ is a left-biased merge operator. The Merkle tree structure encodes this ordering and compositionality.

D.3 Finalization as Closure and Sealing

Finalization ensures that a module is fully resolved: all parents are finalized, and its ancestry is fixed. This is analogous to *closing* a mixin or *sealing* a module in ML. Once finalized, a module cannot change without altering its hash.

D.4 Symbol Resolution as Deterministic Traversal

Symbol resolution is a deterministic traversal over a DAG of finalized modules, following left-to-right override semantics. This can be captured using a judgment of the form:

$$M \vdash x \Downarrow v$$

where M is the finalized module, x is the symbol, and v is the resolved definition. The resolution trace ensures v was not shadowed by any earlier override.

D.5 Typing-Like Guarantees from Hashing Discipline

Although this system does not feature a type system per se, the Merkle-based finalization and override rules yields *determinism*, *compositionality*, and *noninterference* guarantees similar to type safety properties in standard module calculi.

D.6 Summary

This design can be viewed as a sealed, hash-consistent variant of a mixin module calculus, with inheritance DAGs replacing flat composition chains and with hash-based determinism in place of type-based abstraction.

E Formal Lean Proofs

Placeholder Text

F Benchmarks and Evaluation Methodology

This appendix provides full details on the synthetic benchmark methodology, test scenarios, and comparative metrics used to evaluate the Merkle tree-based module system discussed in the main paper.

F.1 Synthetic Evaluation

To validate the model's feasibility, I constructed synthetic benchmarks over simulated module graphs with controlled topologies:

- **Linear chains:** Deep ancestry resolution.
- **Wide graphs:** Shared base modules for deduplication tests.
- **Diamond graphs:** Shadowing and conflict resolution.

Each module contains N symbols and inherits from k parents, with $N \in [10, 1000]$ and $k \in [1, 10]$. Benchmarked operations include:

- **Finalization time:** Merkle root and inheritance trace computation.
- **Symbol resolution:** Lookup and latency analysis.
- **Graph compression:** Deduplication via structural sharing.

Disclaimer. These tests were conducted using a simulated interpreter, not a full runtime. Results are indicative of theoretical scalability, not real-world performance.

F.2 Results Summary

- Finalization scales linearly with symbol count and ancestry depth.
- Symbol lookup remains sublinear in common cases due to caching and reuse.
- Structural sharing yields 60–80% graph compression in shared-ancestor cases.

F.3 Prototype Implementation Plan

A prototype is planned in `Rust`, using:

- `libp2p` for decentralized lookup and DHT-based resolution.
- `Wasmer` to evaluate WebAssembly-based modules and execute finalization logic.

Tangl is one such prototype implementation of this model and will be used to empirically validate performance in future work.

F.4 Planned Evaluation Categories

- **Scalability:** Nested resolution, wide fan-out.
- **Fault tolerance:** Stale caches, resolution under failure.
- **Security:** Malicious ancestry, forked resolution graphs.
- **Tooling usability:** CLI/IDE feedback, trace inspection.
- **Workflow validation:** CI rebuilds, development diffs.

F.5 Comparison to Existing Systems

Tangl will be benchmarked against `npm`, `Cargo`, and `pip` on tasks including:

- Cold vs. warm start resolution.
- Incremental rebuilds.
- Registry fetch latency.

Metrics will include:

- DHT hops per module.
- Cache hit/miss ratios.

- Fallback success under partial connectivity.

G CI/CD and Version Control Integration

This appendix illustrates how the system can integrate into real-world workflows, using Tangl as an example implementation.

G.1 CI/CD Pipelines

Tangl enables efficient CI/CD by leveraging Merkle roots for incremental validation. Because finalization captures full ancestry, CI systems can:

- Cache Merkle roots of finalized subgraphs.
- Skip recomputation when inheritance trees are unchanged.
- Identify resolution-affecting changes early in the build pipeline.

G.2 Version Control and Tagging

Ancestry DAGs can be embedded in Git tags or commit annotations. This allows:

- Cryptographic validation of ancestry consistency.
- Detection of semantic changes from resolution graph diffs.
- Structural tagging schemes for symbolic reproducibility.

G.3 Tooling Extensions

Future versions of Tangl may support:

- IDE visualizations of symbol overrides.
- Git hooks for ancestry change detection.
- CI runners that enforce deterministic finalization traces.

These features position Tangl as a prototype that demonstrates the viability of secure, decentralized module resolution in modern software development workflows.

H Finalization Process Pseudocode

The following pseudocode outlines the finalization process for symbol resolution, type metadata collection, and automatic weakening detection, as used during module finalization.

Listing 3. Finalization with Type Metadata and Weakening Detection

```
def finalize_module(module):
    symbol_table = {}
    type_metadata = {}
    override_metadata = {}

    queue = [module.root_node]
    visited = set()

    while queue:
        curr_node = queue.pop(0)
        visited.add(curr_node)

        for symbol in curr_node.symbols:
            key = (symbol.name, symbol.scope)

            if key not in symbol_table:
                symbol_table[key] = symbol
                type_metadata[key] = symbol.type
                override_metadata[key] = [curr_node.id]
            else:
                # Allow complete replacement
                symbol_table[key] = symbol
                type_metadata[key] = symbol.type
                override_metadata[key].append(curr_node.id)

        for parent in curr_node.parents:
            if parent not in visited:
                queue.append(parent)

    return symbol_table, type_metadata, override_metadata
```